

AUTOCAPNET CODING

```
import streamlit as st

from PIL import Image

from transformers import BlipProcessor, BlipForConditionalGeneration

from deep_translator import GoogleTranslator

from keybert import KeyBERT

from fpdf import FPDF # fpdf2 engine

import torch


# ----- UI CONFIG (MUST BE FIRST STREAMLIT CALL) -----
st.set_page_config(page_title="AutoCapNet Free", layout="centered")


# ----- Load Models -----

@st.cache_resource
def load_models():

    processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")

    model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

    kw_model = KeyBERT()

    return processor, model, kw_model


processor, model, kw_model = load_models()


# ----- UI -----

st.title("🖼️ AutoCapNet")

st.caption("Offline Multilingual Image Captioning & Semantic Tagging")
```

```
language = st.selectbox("Select Language", ["English", "Tamil", "Hindi"])
uploaded_file = st.file_uploader("Upload Image", type=["jpg", "jpeg", "png"])
```

```
# ----- PDF Generator -----
```

```
def create_pdf(text):
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_page()
    pdf.set_margins(15, 15, 15)

    # Unicode font (MANDATORY)
    pdf.add_font("DejaVu", "", "DejaVuSans.ttf", uni=True)
    pdf.set_font("DejaVu", size=12)
```

```
page_width = pdf.w - 2 * pdf.l_margin
```

```
for line in text.split("\n"):
    if line.strip() == "":
        pdf.ln(6)
    else:
        pdf.multi_cell(page_width, 8, line, align="L")
```

```
#  fpdf2 safe output
return bytes(pdf.output(dest="S"))
```

```
# ----- Process -----
```

```
if uploaded_file:
    image = Image.open(uploaded_file).convert("RGB")
```

```

st.image(image, use_column_width=True)

if st.button("Generate"):
    with st.spinner("Processing with AI models..."):

        # Image Captioning
        inputs = processor(image, return_tensors="pt")
        out = model.generate(**inputs, max_new_tokens=50)
        caption_en = processor.decode(out[0], skip_special_tokens=True)

        # Semantic Tags
        keywords = kw_model.extract_keywords(
            caption_en,
            keyphrase_ngram_range=(1, 2),
            stop_words="english",
            top_n=8
        )
        tags_en = [k[0] for k in keywords]

        # Translation
        if language == "Tamil":
            caption = GoogleTranslator(source="en", target="ta").translate(caption_en)
            tags = [GoogleTranslator(source="en", target="ta").translate(t) for t in tags_en]

        elif language == "Hindi":
            caption = GoogleTranslator(source="en", target="hi").translate(caption_en)
            tags = [GoogleTranslator(source="en", target="hi").translate(t) for t in tags_en]

```

```
else:
```

```
    caption = caption_en
```

```
    tags = tags_en
```

```
result = f"Caption:\n{caption}\n\nTags:\n{' '.join(tags)}"
```

```
# ----- UI Output -----
```

```
st.text_area("Result", result, height=220)
```